Lab Report

Drug Review Classification with Custom LSTM Cells

Aouami Salma
Student ID: 127443

Zineb Arrami
Student ID: 118789

Hiba Arbaoui
Student ID: 120401

Ayyadi Marwa
Student ID: 120278

Yahya Abou Elaziz
Student ID: 119993

Hanae Ouazzani Amri
Student ID: 119276

Encadré par :
Alaa Zniber

Module :
Advanced Natural Language Processing

Code : github.com/hiiba8/drug-review-lstm

Année Universitaire : 2026 – 2027

Contents

1 Goal of the task **2**

2 Exploratory data analysis **2**

3 Modeling approach **3**
3.1 The three cells . . . 3
3.2 Training protocol . . . 3

4 Hyperparameter choices **3**

5 Final performance and evaluation **4**
5.1 Comparing the cells . . . 4
5.2 Learning curves . . . 5
5.3 Errors of the best model . . . 6

6 Limitations and possible improvements **6**

Conclusion **6**

1 Goal of the task

We read a patient's review of a drug and predict how they felt about it. The Drugs.com dataset gives a rating from 1 to 10, which we turn into three classes:

- **Negative**: rating ≤ 4
- **Neutral**: rating 5 or 6
- **Positive**: rating ≥ 7

We built every LSTM cell ourselves with `nn.Linear` (no `nn.LSTM`): a vanilla LSTM as baseline, plus two variants, **CIFG** and **LSTMP**. Then we studied how learning rate, optimizer, batch size and initialization change the results.

2 Exploratory data analysis

The dataset comes with an official train/test split. We keep the test set as it is and take a stratified 10% of the training set for validation. The same split is used by every model: **145,167** training, **16,130** validation and **53,766** test reviews.

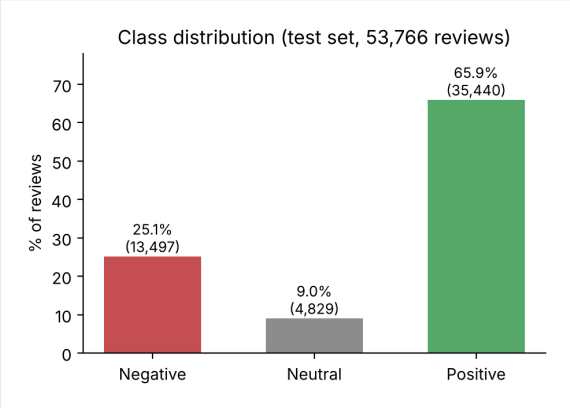


Figure 1: Class distribution (test set)

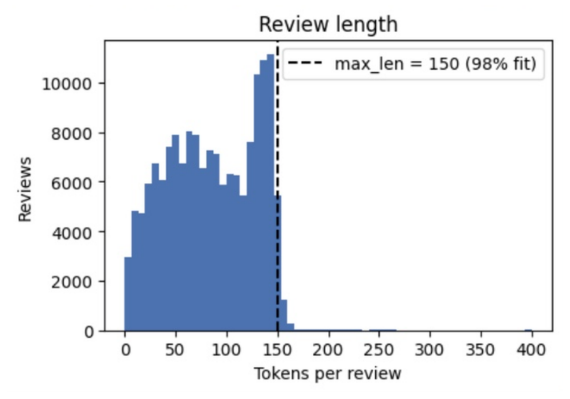


Figure 2: Number of tokens per review

The data is very unbalanced: 65.9% of test reviews are positive, 25.1% negative and only 9.0% neutral (the training set has the same shape). Because of this we use **macro-F1** as our main metric (accuracy would reward a model that just says "positive"), and we weight the loss by the inverse class frequency.

Reviews are fairly short: 84.7 tokens on average (median 84). The peak just below 150 tokens probably comes from the length limit on Drugs.com. We cut reviews at 150 tokens, which keeps 98% of them whole while keeping training time reasonable. The text was lowercased, HTML entities were removed, and we kept the 20,000 most frequent words.

3 Modeling approach

All models share the same pipeline: embedding (128) → recurrent cell (hidden size 128) → hidden state of the last real token → dropout (0.3) → linear layer with 3 outputs. Only the cell changes.

3.1 The three cells

Vanilla LSTM. The standard cell with input, forget and output gates. For speed we compute the four gates with one `nn.Linear` on x_t and one on h_{t-1}, then split the result. This is mathematically the same as eight separate matrices.

CIFG (Coupled Input–Forget Gate). The input gate is replaced by $i_t = 1 - f_t$: the cell forgets exactly as much as it writes. This removes one gate, so 25% fewer cell parameters.

LSTMP (LSTM with projection). The output m_t is projected to a smaller size: $h_t = W_p m_t + b_p$, with $P = 64 < H = 128$. The recurrent connection uses this smaller h_t.

```
class CIFGCell(nn.Module):                # i = 1 - f
    def forward(self, x, state):
        h, c = state
        f, o, g = (self.x2g(x) + self.h2g(h)).chunk(3, -1)
        f, o, g = torch.sigmoid(f), torch.sigmoid(o), torch.tanh(g)
        c = f * c + (1 - f) * g
        return o * torch.tanh(c), c
```

We also implemented the **WMC** cell (working memory connections), but did not have time to train it.

3.2 Training protocol

Every run uses the same splits and the same evaluation: we train for 8 epochs, keep the epoch with the best **validation** macro-F1, and only then evaluate once on the test set. Gradients are clipped at 1.0. All runs were done on a Colab T4 GPU (about 35 minutes each).

4 Hyperparameter choices

We started from a reference setting (learning rate 10^{-3}, Adam, batch size 64, PyTorch default initialization) and changed **one thing at a time** on the vanilla LSTM.

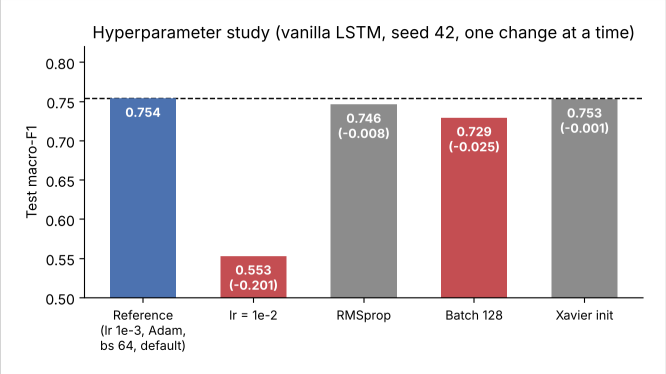


Figure 3: Test macro-F1 for each change

Change	Val F1	Test F1
Reference	0.753	0.754
lr 10^{-2}	0.548	0.553
RMSprop	0.746	0.746
Batch 128	0.731	0.729
Xavier init	0.750	0.753

Table 1: Hyperparameter study (vanilla, seed 42)

Observation

Learning rate is by far the most important setting: at 10^{-2} training becomes unstable: the training loss stops decreasing after epoch 3 and then goes back up. Batch size 128 also scores lower, but with a fixed 8 epochs it gets half as many updates, so it is probably just undertrained. RMSprop and Xavier make no real difference. We kept the reference setting for all models.

5 Final performance and evaluation

5.1 Comparing the cells

Model	Cell params	Total params	Seeds	Test acc	Test macro-F1
Vanilla LSTM	131,584	2,691,971	2	0.846	0.756
CIFG	**98,688**	2,659,075	2	**0.849**	**0.761**
LSTMP	107,072	2,667,267	1	0.842	0.749

Table 2: Architecture comparison (mean over seeds). Most parameters are in the embedding layer, which is identical for all models.

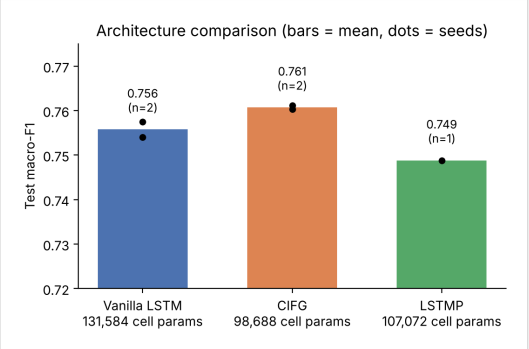


Figure 4: Test macro-F1 per cell (bar = mean, dots = individual seeds)

Observation

CIFG is slightly better than the vanilla LSTM while using 25% fewer cell parameters. The gap is small (about 0.005 F1, close to the difference between two seeds), so we read it as "at least as good", not as a clear win. LSTMP is a little behind, which makes sense: it squeezes the state into 64 dimensions.

5.2 Learning curves

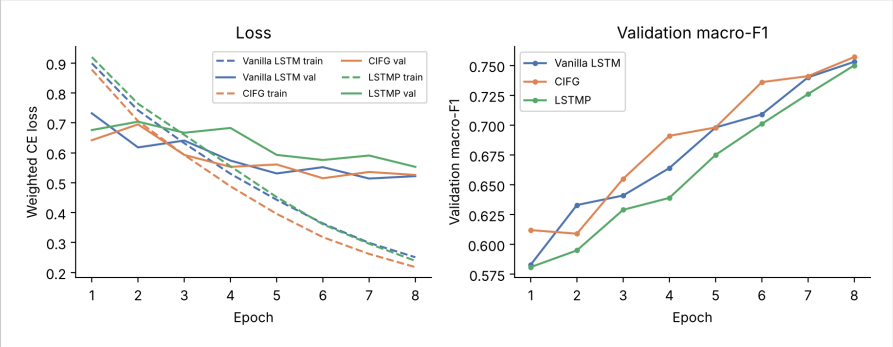


Figure 5: Loss (left) and validation macro-F1 (right) for the three cells

Validation F1 is still going up at epoch 8 for all three cells, so the models are not fully trained yet. The validation loss flattens out after epoch 5 while validation F1 keeps rising. (Training loss is class-weighted and validation loss is not, so the two curves are not directly comparable.)

5.3 Errors of the best model

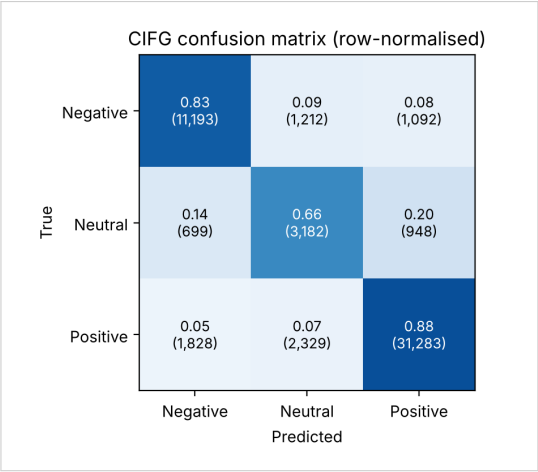


Figure 6: CIFG confusion matrix (test, row-normalised)

Recall	Neg.	Neutral	Pos.
Vanilla	0.846	0.616	0.876
CIFG	0.829	**0.659**	0.883
LSTMP	0.813	0.626	0.882

Table 3: Recall per class on the test set (seed 42)

Observation

Neutral is the hardest class (about 66% recall). When the model misses a neutral review it usually calls it positive. This is not surprising: a 5 or 6 out of 10 often reads like a mixed or mildly positive review.

6 Limitations and possible improvements

- **Train longer, with more seeds.** Validation F1 was still rising at epoch 8, and we only have 1–2 seeds per model. More epochs with early stopping, and more seeds, would tell us if the small gaps are real.
- **Wider hyperparameter search.** We only tried one other learning rate (10^{-2}); a smaller one (10^{-4}) and a learning-rate scheduler are worth testing. Batch size should be compared with the same number of updates.
- **Train the WMC cell**, which is already implemented.
- **Better inputs.** Pre-trained word embeddings (GloVe, fastText) or a bidirectional cell could help the model understand mixed reviews, which is exactly where the neutral class fails.

Conclusion

All three cells reach about 0.75 macro-F1 and 85% accuracy. **CIFG** did best (0.761) with the fewest parameters, so the input gate can be dropped without losing performance. The learning rate mattered far more than the choice of cell.